

DevOps Interview Preparation Series (Day 3)

 by T Keerti

In the world of DevOps, scripting is a core skill that enables automation, configuration, and system management. Two popular scripting choices are Shell scripting and Python. While both are powerful, understanding when to use which can significantly improve efficiency and maintainability in your workflows.

How They Interact with the System

- Shell:

Native to Linux. Directly interacts with the operating system.

Flow: UI → Shell/CMD → Kernel

- Python:

More abstracted. Often used to interact with systems through shell commands and APIs.

Flow: UI → Python → Shell/CMD → Kernel

When to Use Shell?

- Best for native automation on Linux servers.
- Ideal for lightweight scripting tasks like backups, cron jobs, or startup tasks.
- Consumes less resources (RAM, CPU, Disk).
- Requires less time for simple tasks.

Shell is your go-to when automating system-level tasks directly within the Linux environment.

When to Use Python?

- Excellent for cross-platform scripting and integration with external systems or APIs.
- Great for writing modular, scalable code.
- Supports object-oriented features and complex logic structures.
- Strong ecosystem for cloud, monitoring, and DevOps tools (e.g., Boto3 for AWS, requests, etc.).

Use Python when you're building robust scripts that involve external APIs, complex logic, or integrations beyond the OS.

`#!/bin/bash`

`#!` → tells the OS which interpreter to use.

`/bin/bash` → path of bash shell.

When you run the script, Linux uses bash to execute it.

1. Hello world : echo "Hello, world!"
2. Count non-hidden files in current directory : ls -l | wc -l

| wc -l — pipe to wc -l to count the number of lines
3. Count lines in a file : wc -l < filename
4. Show top 5 CPU processes : ps -ef pid,ppid,cmd,%cpu --sort=-%cpu | head -n 6
ps -eo pid,ppid,cmd,%cpu — list processes with PID, PPID, command, and CPU usage.

--sort=-%cpu — sort descending by CPU percent.

| head -n 6 — show the header + top 5 lines (or top 6 including header).
- 5.Find files modified in last N days : find /path -type f -mtime -7
list files modified within last 7 days under /path.

6.One-line backup (tar & gzip) : tar -czf backup-\$(date +%F).tgz /path/to/dir
tar -czf — create (-c), gzip (-z), and write to file (-f).
backup-\$(date +%F).tgz — filename with current date YYYY-MM-DD.
/path/to/dir — directory to archive.

7. Disk usage alert script

```
#!/bin/bash
df -h | awk 'NR>1 && int($5) > 80 {print $6 " is " $5 " full"}'
NR>1 : skip the first line (the header).
$5: column 5 is the "Use%" value (e.g., 85%).
int($5) → removes the % sign and converts to number.
> 80 → checks if usage is more than 80%.
print $6 " is " $5 " full" → prints the mount point and usage.
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	20G	15G	5.0G	75%	/
tmpfs	1000M	10M	990M	1%	/run
/dev/sdb2	100G	95G	5.0G	95%	/home

8.Special Variables in Shell Scripting

- \$0: Script name
- \$1, \$2...: Positional arguments
- \$@: All arguments individually
- *: All arguments as one string
- #: Number of arguments passed to the script
- \$PWD: Current working directory
- \$HOME: Home directory of the current user
- \$USER: User running the script
- \$\$: Process ID of the current script
- !\$: PID of the last background command
- ?: Exit status of the last executed comand
- :-: Current shell flags/options

9.Check if a file exists

```
#!/bin/bash
file="$1"
if [ -f "$file" ];
then echo "File exists: $file"
else echo "File not found: $file" fi
```

file="\$1" — first argument.
-f tests regular file existence.
echo prints result.

10. Check if directory exists and create if missing

```
#!/bin/bash
dir="$1"
if [ -d "$dir" ];
then echo "Directory exists: $dir"
else mkdir -p "$dir"
echo "Directory created: $dir" fi
```

-d checks directory.
mkdir -p creates directory and parents if needed.

11. Loop through files in directory

```
#!/bin/bash
dir="{1:-.}"
for f in "$dir"/*;
do [ -e "$f" ] || continue
echo "Found: $f" done
```

{1:-.} default to current dir.
Loop iterates entries; [-e] ensures skip when no matches.

12. Count lines in a file

```
#!/bin/bash
file="$1"
if [ ! -f "$file" ];
then
echo "Usage: $0 file" >&2
exit 1
fi
wc -l < "$file"
```

wc -l prints number of lines; < avoids printing filename.

13.Simple backup (tar gz)

```
#!/bin/bash
src="{1:-.}"
dest="{2:-/tmp/backup-$(date +%F).tar.gz}"
tar -czf "$dest" -C "$(dirname "$src")" "$(basename "$src")"
echo "Saved: $dest"
```

Create gzip tar of src; default destination uses today's date.

14.Check CPU Usage

```
#!/bin/bash
top -bn1 | grep "Cpu(s)" | awk '{print 100 - $8"% used"}'
```

top -bn1 → runs top once in batch mode (no live screen).
grep "Cpu(s)" → finds the line showing CPU usage.
\$8 → is the **idle percentage**.
100 - \$8 → gives CPU used percentage.

15. Check Memory Usage

```
#!/bin/bash
free -m | awk 'NR==2 {printf "Memory used: %.2f%%\n", $3*100/$2}'
```

free -m → shows memory in MB.
NR==2 → selects the second line (main memory).
\$3 = used memory, \$2 = total memory.
Formula = (used / total) × 100.

16. Find Large Files : find / -type f -size +100M 2>/dev/null | head
find / -type f -size +100M → finds files larger than 100 MB.
2>/dev/null → hides permission errors.
head → shows first few for speed.

Task	Command Used	Concept
Disk usage check	df , awk	Filesystem usage
CPU usage	top , awk	CPU idle vs used
Memory usage	free , awk	Memory % used
Top 5 memory	ps , head	Process sort
Top 5 CPU	ps , head	Process sort
Large files	find	File size
Service check	systemctl	Active or not
Logged users	who , wc	Count
Total users	grep , /etc/passwd	User accounts
Uptime	uptime -p	System duration

17. Email alert using sendmail

```
#!/bin/bash
to="$1"; subj="$2"; body="$3"
{
echo "Subject: $subj"
echo
echo "$body"
} | sendmail "$to"
```

Pipes email headers/body to sendmail for delivery.

18. Rotate a log file (simple)

```
#!/bin/bash
log="/var/log/myapp.log"
if [ -f "$log" ]; then
mv "$log" "${log}.${date +%F-%H%M%S}"
: > "$log"
echo "Rotated $log"
fi
```

mv renames current log with timestamp; : > "\$log" truncates/creates new file.

19. Read file line-by-line safely

```
#!/bin/bash
file="$1"
while IFS= read -r line; do
echo "Line: $line"
done < "$file"
```

IFS= preserves leading/trailing whitespace; -r prevents backslash escapes.
Redirect file into while to read each line.

20. Check if a command exists and suggest install

```
#!/bin/bash
cmd="$1"
if command -v "$cmd" >/dev/null 2>&1; then
echo "$cmd is installed"
else
echo "$cmd not found. Try: sudo apt-get install $cmd"
fi
```

command -v checks availability; suggests apt install

21.How do you make a shell script executable : chmod +x script.sh

22. How do you run a shell script?
./script.sh → run with current shell.
bash script.sh → explicitly run with bash.
sh script.sh → run with sh (less preferred if bash-specific syntax).

23. How do you pass arguments to a script : ./script.sh arg1 arg2

```
echo $1
echo $2
```

24. How do you read user input

```
read name
echo "Hello, $name"
```

read waits for input.
Value is stored in name.

25. What does read -p do?

```
read -p "Enter your name: " user
echo "Welcome $user"
```

26. What is the use of export : export DB_USER=admin

27. How do you debug a shell script?

Use bash -x script.sh or add:

```
set -x # enable debug
set +x # disable debug
```

28. How do you write an if-else block

```
if [ $x -gt 10] ; then
echo "Big"
else
echo "Small"
fi
```

29. How to check if a directory exists?

```
if [ -d "$dir" ]; then
echo "Directory exists"
fi
```

30. How do you compare two numbers?

```
if [ $a -gt $b ]; then
echo "A > B"
fi
```

31. Iterate numbers 1 to 10

```
for i in {1..10}; do
echo "Count $i"
done
```

32. Use of until loop

```
i=0
until [ $i -ge 5 ]; do
echo "$i"
((i++))
done
```

33. What does > and >> do in shell

> → redirects output to a file (overwrite).

>> → appends output to a file (does not overwrite).

```
echo "Hello" > file.txt # replaces content
echo "World" >> file.txt # appends
```

34. How to transfer a file between servers: scp file.txt user@server:/tmp/

35. How to use tee to log and continue streaming output : cmd | tee -a /var/log/myapp.log
tee -a → append output to logfile while still printing to stdout.

Shell Scripting Quick Reference

- Make executable: `chmod +x script.sh` → run `./script.sh` or `bash script.sh`.
- Positional args: `$0` (script), `$1..$n`; count with `$#`; all as list: `"$@"`.
- Special: `$?` (last exit), `$$` (this PID), `#!` (last bg PID), `$SHELL`, `$USER`, `$HOME`, `$PWD`.
- Debugging: `set -x` (on), `set +x` (off), or run `bash -x script.sh`.

Conditionals & Loops

- If: `if [ condition ]; then ...; elif ...; else ...; fi`.
- Numeric: `-eq -ne -gt -lt -ge -le`. String: `= !=` or `==` in `[[ ]]`.
- For loop: `for i in {1..5}; do ...; done`
- While: `while [ cond ]; do ...; done`
- Read file: `while IFS= read -r line; do ...; done < file`

I/O, Redirection, Pipes

- Overwrite `>` | append `>>` | stderr `2>` | merge `2>&1`.
- Here-doc: `cat <<EOF >file ... EOF`.

Process & Job Control

- Background: `cmd &` → list jobs → `fg %1` → `bg %1`.
- `ps -eo pid,ppid,cmd,%cpu --sort=-%cpu | head` → top CPU.
- Kill: `pkill name` or `kill -9 PID`.

Files & Search

- Find recent: `find /path -type f -mtime -7`
- Large files: `find / -type f -size +100M 2>/dev/null`
- Disk: `df -h` (partitions), `du -sh dir` (dir size).

Text tools

- `awk '{print $1}'` → columns, `awk` for logic.
- `sed` → stream edit, `grep` → search; `grep -i` ignore case.

Monitoring & Alerts

- Tail logs: `tail -f /var/log/syslog | grep --line-buffered ERROR`
- Disk alert one-liner: `df -h | awk 'NR>1 && int($5) > 80 {print $6 " is " $5 " full"}'`
- Cron: `crontab -e` — example entry: `0 2 * * * /root/backup.sh`

Shell Script 01

🚀 Automate User Information Extraction from /etc/passwd with Bash!

Here's how you can effortlessly parse the /etc/passwd file to extract user details with a simple Bash script. Let's dive into the steps!

💡 1 Define the File Path

- ◆ Begin by specifying the file to process: /etc/passwd.

💡 2 Check if the File Exists

- ◆ The script ensures the file exists before processing.
- ◆ If it doesn't, you get an alert: "File doesn't exist!".

💡 3 Process Each Line

- ◆ Use a while loop with IFS=":" to split each line into clear fields.
- ◆ This approach makes the data easy to read and process.

💡 4 Output Relevant Fields

- ◆ Extracts and prints:

👤 Username

🔑 Password placeholder

ID User ID

👥 Group ID

This script is a simple, yet powerful way to automate user information retrieval.

✨ Why Use This Script?

- ✓ Automates tedious tasks
- ✓ Improves data clarity
- ✓ Simplifies user management

```
#!/bin/bash
# Define the file path
file=/etc/passwd

# Check if the file exists
if [ ! -f "$file" ]; then
    echo "Source file: $file doesn't exist."
    exit 1
fi

# Process the file line by line
while IFS=":" read -r username password user_id group_id; do
    echo "Username: $username"
    echo "Password: $password"
    echo "User ID: $user_id"
    echo "Group ID: $group_id"
done < "$file"

# IFS (Internal Field Separator) splits fields by ":"
# Script extracts and displays user details from /etc/passwd
```

Shell Script 02

🚀 Automating Software Setup with Bash Scripting! 🛠️

👋 Hey LinkedIn Network,

Today, I want to share a cool Bash script that automates the installation of key tools like MySQL, Git, and Docker on Linux systems. 🖥️ This script ensures a seamless setup process, validating each step for success or failure.

Here's a high-level look at the script's flow:

- 1 User Validation: Checks if the script is executed with root access.
- 2 Error Handling: Uses a VALIDATE function to log success or failure for each command.
- 3 Automation: Automatically installs essential tools while keeping logs for troubleshooting.

🚀 Automating Software Setup with Bash Scripting! 🛠️

👋 Hey LinkedIn Network,

Today, I want to share a cool Bash script that automates the installation of key tools like MySQL, Git, and Docker on Linux systems. 🖥️ This script ensures a seamless setup process, validating each step for success or failure.

🔑 Pro Tip: Always log the outputs of your commands (LOGFILE), so you can debug issues if things go south!

```
#!/bin/bash

USERID=$(id -u)
TIMESTAMP=$(date +%F-%H-%M-%S)
SCRIPT_NAME=$(echo $0 | cut -d "/" -f1)
LOGFILE="/tmp/${SCRIPT_NAME}-${TIMESTAMP}.log"
R="\e[31m"
G="\e[32m"
N="\e[0m"

echo "Script started executing at: $TIMESTAMP"

VALIDATE() {
    if [ $1 -ne 0 ]; then
        echo -e "$2...${R}FAILURE${N}"
        exit 1
    else
        echo -e "$2...${G}SUCCESS${N}"
    fi
}

if [ $USERID -ne 0 ]; then
    echo "Please run this script with root access."
    exit 1
fi
```

Shell Script 03

Automate CPU Usage Monitoring with a Bash Script!

In the world of DevOps, automation is key to ensuring system stability.

Here's a simple Bash script I created to monitor CPU usage for a specific process and take action when thresholds are exceeded.

 What does this script do?

 Monitors a process by name.

 Checks its CPU usage.

Why use this?

- ✓ Keeps resource-hogging processes in check.
- ✓ Saves time and ensures efficient resource utilization.
- ✓ A handy solution for real-time monitoring.

```
#!/bin/bash
# Process name to monitor
process_name="your_process_name_here"

# CPU usage threshold (in percentage)
threshold=50

# Get PID of the process
pid=$(ps aux | grep "$process_name" | grep -v grep | awk '{print $2}')

if [ -z "$pid" ]; then
    echo "Process not found."
else
    # Get current CPU usage of the process
    cpu_usage=$(ps -p "$pid" -o %cpu | awk 'NR>1' | awk -F. '{print $1}')
    echo "Current CPU usage of $process_name (PID $pid): $cpu_usage%"

    # Compare CPU usage with threshold
    if [ "$cpu_usage" -gt "$threshold" ]; then
        echo "CPU usage exceeds threshold. Terminating process..."
        # Terminate the process
        kill "$pid"
        echo "Process terminated."
    else
        echo "CPU usage within threshold."
    fi
fi
```

Shell Script 04

Automate Log Cleanup with Bash Script

 Title: Automate Log Cleanup with Bash Script

 Explanation: Streamline your system maintenance with this handy Bash script! This script helps automate the process of cleaning up log files by:

- 1 Verifying if the source directory exists.
- 2 Identifying .log files older than 14 days.
- 3 Safely deleting the outdated files.

Use this to keep your servers clean and efficient with minimal manual intervention.

 Script:

Start automating today and save valuable time!

 Pro tip: Schedule this script as a Cron job to run periodically and never worry about log accumulation again.

```
#!/bin/bash

SOURCE_DIRECTORY=/tmp/app-logs
R="\e[31m"
G="\e[32m"
Y="\e[33m"
N="\e[0m"

if [ -d "$SOURCE_DIRECTORY" ]; then
    echo -e "${G}Source directory exists${N}"
else
    echo -e "${R}Please make sure
${SOURCE_DIRECTORY} exists${N}"
    exit 1
fi

FILES=$(find "$SOURCE_DIRECTORY" -name "*.log" -
mtime +14)

while IFS= read -r line; do
    echo "Deleting file: $line"
    rm -rf "$line"
done <<< "$FILES"
```

Shell Script 05

Automating Disk Usage Alerts with Bash Scripting

 Do you want to proactively monitor your system's disk usage and receive alerts when it crosses a threshold? Here's a simple Bash script to automate this process and send email notifications!

What Does This Script Do?

- 1 Checks disk usage for xfs file systems.
- 2 Compares usage percentages against a predefined threshold (6% in this case).
- 3 Sends a customizable email notification if usage exceeds the threshold.

Why is this useful?

This ensures you never run into unexpected storage issues on critical servers. A simple script like this can save hours of troubleshooting!

Explanation of the Script:

`df -hT`: Lists disk space usage in human-readable format along with the filesystem type.

`grep xfs`: Filters results to only include xfs file systems.

`awk` and `cut`: Parse the disk usage percentage and folder paths.

`if condition`: Compares the usage percentage against the threshold.

`mail command`: Sends an email alert to the specified address with the folder and usage details.

```
#!/bin/bash

DISK_USAGE=$(df -hT | grep xfs)
THRESHOLD=6
MESSAGE=""

while IFS= read -r line; do
    USAGE=$(echo $line | awk -F " " '{print $6}' | cut -d "%" -f1)
    FOLDER=$(echo $line | awk -F " " '{print $NF}')
    if [ $USAGE -ge $THRESHOLD ]; then
        MESSAGE+="$FOLDER is more than $THRESHOLD%, Current usage: $USAGE%\n"
    fi
done <<< "$DISK_USAGE"

TIMESTAMP=$(date '+%Y-%m-%d %H:%M:%S')

echo -e "Message: $MESSAGE"
echo -e "Disk Usage Alert - $TIMESTAMP\n\n$MESSAGE" | mail -s "Disk Usage Alert" k@gmail.com
```

🎯 Automate Package Installation with Bash 🚀

Hey network! 🙌

Are you managing Linux systems and tired of manually installing packages? Here's a simple yet effective Bash script .It validates installations, logs results, and ensures smooth operations—all while keeping things professional. 💡

📄 Highlights:

- ✅ Automatically checks if packages are already installed
- ✅ Installs missing packages with validation and error handling
- ✅ Generates a log file with timestamps for debugging

Here's the script for you to try! 🛠️

📌 How to Use:

- 1 Copy the script to a .sh file (e.g., [package_installer.sh](#)).
- 2 Make it executable: `chmod +x package\_installer.sh`.
- 3 Run it with root privileges and pass package names as arguments:

Replace dnf with apt, yum, or your package manager of choice for compatibility.

```
#!/bin/bash

# Get the current user ID and timestamp
USERID=$(id -u)
TIMESTAMP=$(date +%F-%H-%M-%S)
SCRIPT_NAME=$(basename $0)
LOGFILE=/tmp/${SCRIPT_NAME}-${TIMESTAMP}.log

# Define colors for output
R="\e[31m" # Red color for FAILURE
G="\e[32m" # Green color for SUCCESS
Y="\e[33m" # Yellow color for SKIPPING
N="\e[0m" # Reset color

# Function to validate command success
VALIDATE() {
    if [ $1 -ne 0 ]; then
        echo -e "${R}$2${N}"
        exit 1
    else
        echo -e "${G}$2${N}"
    fi
}

# Check if user is root
if [ $USERID -ne 0 ]; then
    echo "Please run this script with root access."
    exit 1
else
    echo "You are superuser."
fi

# Install each package provided as an argument
for i in "$@"; do
    echo "Package to install: $i"
    dnf list installed $i &>> $LOGFILE
    if [ $? -eq 0 ]; then
        echo -e "${Y}$i already installed...${Y}${SKIPPING}${N}"
    else
        dnf install $i -y &>> $LOGFILE
        VALIDATE $? "Installation of $i"
    fi
done
```

🚀 Automating Email Alerts with a Bash Script! 🔧

Simple but powerful Bash script for sending customizable email alerts [🔗](#). Let's break it down step by step and understand its purpose and functionality:

This script saves time ⌚ and ensures consistency across alert emails.

- Input Arguments:
- TO_TEAM: Team to notify (used to replace the placeholder in the template).
 - ALERT_TYPE: The type of alert (e.g., Critical, Warning).
 - BODY: The main content of the alert message.
 - TO_ADDRESS: The recipient's email address.
 - SUBJECT: The subject of the email.

```
#!/bin/bash

# Email Notification Script

# Arguments:
# $1 - TO_TEAM: Team to notify (placeholder in template)
# $2 - ALERT_TYPE: Type of alert (e.g., Critical, Warning)
# $3 - BODY: Main content of the alert message
# $4 - TO_ADDRESS: Recipient email address
# $5 - SUBJECT: Email subject

TO_TEAM=$1
ALERT_TYPE=$2
BODY=$3

# Escape special characters in the BODY content
ESCAPE_BODY=$(printf '%s\n' "$BODY" | sed -e 's/11\\$*-^11/11819/')

TO_ADDRESS=$4
SUBJECT=$5

# Replace placeholders in the HTML template
FINAL_BODY=$(sed -e "s/TO_TEAM/$TO_TEAM/g" \
                 -e "s/ALERT_TYPE/$ALERT_TYPE/g" \
                 -e "s/BODY/$ESCAPE_BODY/g"
19.Mail_template.html)

# Send the email using the 'mail' command
echo "$FINAL_BODY" | mail -s "$(echo -e "$SUBJECT\nContent-Type: text/html")"
"$TO_ADDRESS"
```